

面向不确定性的 DAG 调度*

Grzegorz Malewicz
Department of Engineering
Google, Inc.
Mountain View, CA 94043, USA
malewicz@google.com

October 5, 2026

摘要

本文提出一种并行调度问题：用有向无环图表示 t 个任务及其依赖关系，并在 n 个不可靠工作者上执行。工作者 i 以概率 $p_{i,j}$ 正确执行任务 j 。目标是找到一个调度方案 Σ ，规定执行过程中如何将工作者分配到任务（可以并行，也可以冗余执行），从而最小化期望完成时间。这个基本并行调度问题出现在网络计算和项目管理领域，并具有若干应用。

我们给出一个多项式时间算法，适用于 DAG 宽度至多为常数、且工作者数量也至多为常数的情况。这两个限制看起来可能过于严格，但它们在根本上是必要的。具体而言，当工作者数量为常数、DAG 宽度可以增长时，该问题为 NP 难；当 DAG 宽度为常数、工作者数量可以增长时，该问题也为 NP 难。如果 DAG 宽度和工作者数量均不受限制，则除非 $P=NP$ ，否则该问题无法在小于 $5/4$ 的近似因子内近似。

关键词：算法与理论，组合优化，网络计算，马尔可夫链，概率故障，项目管理，资源约束，DAG 调度。

1 引言

过去数年，网络计算基础设施不断发展，使计算能够在共享的分布式资源上执行 [10]。由于硬件和软件故障以及共享使用，计算机、磁盘和网络常常变慢或停止工作。尽管如此，人们仍有很大需求在网络上执行任务依赖复杂的科学计算（参见 [1, 2, 6, 16]）。尚待解决的重要挑战之一，是如何使用不可靠资源快速执行大规模复杂计算。当某个任务未能正确完成时，整体进度可能被拖延，因为依赖它的任务必须等到该任务成功执行后才能继续。可以预见，任务依赖关系与资源可靠性对于快速执行计算具有重要作用。因此，我们希望确定这些因素之间的关系，并设计利用不可靠资源快速执行复杂计算的算法。

类似问题也出现在生产规划、软件开发等项目管理中 [14]。这里给定一组活动及其优先关系约束，可以将工作者分配给活动。但实际中，分配到某项活动的工作者可能无法完成它。例如，一项活动包含编写代码和测试，测试可能失败。项目管理者可以根据以往合作经验，估计某个工作者执行某项活动的成功概率，也可以为活动冗余分配多个工作者。例如，两名工作者可以独立编写并测

*本工作的初步版本发表于第 17 届 ACM Symposium on Parallelism in Algorithms and Architectures (SPAA'05) 论文集。部分研究在访问 Argonne National Laboratory 的 Division of Mathematics and Computer Science (Argonne, IL 60439) 期间完成，由 NSF 项目 ITR-800864 资助；另有部分在 University of Alabama 的 Department of Computer Science (Tuscaloosa, AL 35487) 停留期间完成。

试同一段代码，只要至少一个测试成功，该活动就完成。因此，管理者面对的问题是：如何在项目执行过程中将工作者分配到活动，可以并行、可以冗余，从而最小化整个项目的执行时间。

上述两个应用领域促使我们研究如下基本并行计算调度问题。给定有向无环图 (DAG)，表示 t 个任务及其依赖关系；另有 n 个工作者。在每个时间单位，依据优先关系约束和已完成任务，将工作者以某种方式分配到当前“可执行”的任务：每名工作者一次至多分配给一个任务，可以同时分配多个任务，也可以将多名工作者分配给同一任务，并允许工作者空闲。随后，工作者尝试执行分配的任务。工作者 i 对任务 j 的尝试以概率 $0 \leq p_{i,j} \leq 1$ 成功。下一个时间单位重新分配工作者。重复该过程，直到 t 个任务全部完成。目标是确定一个调度方案 Σ ，规定整个执行过程中的分配方式，并最小化期望完成时间。

1.1 本文贡献

本文提出并研究利用不可靠工作者执行 DAG 的并行调度问题，具体贡献如下：

- (i) 提出一个新的组合优化问题：给定包含 t 个任务的 DAG G 和 n 名工作者，其中 $p_{i,j}$ 是工作者 i 正确执行任务 j 的概率，寻找最小化期望完成时间的调度方案 Σ 。

我们说明，该问题能否在多项式时间内求解，关键取决于两个自然参数：工作者数量，以及 DAG G 的宽度。后者是一个反链的最大基数（全部技术术语在第 2 节定义）。

- (ii) 对问题的受限版本，设计一个在多项式时间内找到最优调度方案的算法。具体而言，假设 DAG G 的宽度至多为常数，同时工作者数量也至多为常数。该算法允许 DAG 结构复杂、规模增长，但 DAG 必须“窄”。算法采用动态规划：给定 DAG G ，构造称为执行的^{可容许演化}的有向图 A ，包含调度方案在某一时刻可能已完成的全部任务集合，以及一个集合如何在一个时间单位内由另一个集合产生。该图 A 本身也是 DAG。随后给出递推式，依据节点的子节点定义，为 A 的该节点定义调度方案。
- (iii) 通过复杂度下界说明上述限制从根本上是必要的，与算法结果形成鲜明对应。具体而言，当 DAG 宽度为常数而工作者数量可以增长时，优化问题为 NP 难；当工作者数量为常数而 DAG 宽度可以增长时，也为 NP 难。进一步地，若 DAG 宽度和工作者数量均不受限制，则除非 $P=NP$ ，否则不存在近似因子小于 $5/4$ 的近似算法。

我们的归约表明，该问题的复杂度上升十分迅速。首先，DAG 只有一个任务时，问题很简单：最优方案只需将全部工作者分配给该任务。但 DAG 只有两个独立任务时，问题就已经为 NP 难！（可以说，这是“第二简单”的 DAG。）其次，只有一个工作者时，问题也容易：最优方案沿任意拓扑排序，依次将唯一工作者分配给当前任务即可。但只有两个工作者时，问题就已经为 NP 难！

1.2 相关工作

已有若干研究处理资源受限的调度 [30, 28]。其中许多问题为 NP 难 [5]，研究通常侧重启发式方法。本文与这些研究的区别在于，还考虑资源故障。更接近本文的是文献 [27]：每个任务可采用几种执行方式之一，每种方式具有相应的资源需求、持续时间和失败概率。执行失败后，任务以无故障方式重新执行；目标是^{最小化期望完成时间}，作者提出若干启发式方法。文献 [37] 研究类似模型，但目标是通过启发式方法最大化成功完成概率。

在不确定性项目调度 [15, 8, 9] 中，每个任务的持续时间是随机变量，并规定执行任务所需的不同资源数量。每种资源的可用总量固定，目标是在项目执行过程中确定资源分配方式，最小化期望完成时间。工作者也可视为资源，因此本文问题与之相似；区别在于，我们允许将多个工作者分配

给同一任务，从而改变任务持续时间分布。文献 [38] 研究一种模型，其中 DAG 中每个任务的执行时间为随机变量，其分布依赖分配给该任务的资源量。若执行超过某个阈值，执行可能失败。目标是最大化完成概率，已有若干启发式方法。

随机调度的一个目标是：当任务持续时间为随机变量、且需要在并行机器上调度时，最小化期望完成时间。已有针对独立任务 [19, 13] 和依赖任务 [35] 的研究，但其设置与本文不同，因为我们允许将两台或更多机器分配给同一任务，从而改变任务持续时间的概率分布。其他相关问题包括网络可靠性问题和网络生存性问题 [12]。这些问题也以概率方式建模故障，但优化目标不同。

通过互联网执行任务带来了独特挑战。文献 [11] 已确定：在完成时间受限的条件下，哪些依赖任务应由可靠服务器执行，哪些应由以概率方式建模不可靠性的工作者执行，从而最大化期望正确完成任务数。文献 [34] 针对独立任务研究了类似概率模型。另一个相关问题是如何调度任务，使任务以尽可能高的速率变为可分配给工作者、从而可执行的任务 [32, 33, 25, 24, 23, 3]。

网络计算和项目管理中有若干与本文相关的系统。例如，Condor [36] 执行具有复杂任务依赖关系的计算，而接收任务的客户端通常不可靠。Condor 一次将一个任务分配给一台计算机，不采用冗余，并用“FIFO”拓扑排序安排任务提交。这有时会导致计算资源利用低效。

项目管理系统的一个例子是 Microsoft Project 2003 [26]。该系统可以基于任务持续时间的概率模型，使用 PERT 算法估计项目持续时间，但不考虑资源约束。

在本文的一篇配套研究中，作者探讨了算法实现细节和实验 [22]。具体而言，提出若干理论与实践方法来构造高效实现，并通过实验评估这些方法的益处及实现的可扩展性。

最近，本文问题的若干情形已有近似算法 [20, 21]。独立任务情形可以达到 $O(\log t)$ 近似。更复杂依赖结构也存在近似算法：对互不相交的链，已有 $O(\log n \log t \log(n+t) / \log \log(n+t))$ 近似；一组有向外向树或内向树可达到 $O(\log n \log^2 t)$ 近似，而有向森林可达到 $O(\log n \log^2 t \log(n+t) / \log \log(n+t))$ 近似。

1.3 论文结构

本文余下结构如下。第 2 节定义工作者以给定概率失败时的 DAG 执行模型，并形式化最小化期望完成时间的优化问题。第 3 节给出 DAG 宽度为常数、工作者数量也为常数时的多项式时间算法。最后，第 4 节解释为什么必须同时限制 DAG 宽度和工作者数量。

2 模型与背景

有向图 $\mathcal{G}$ 由节点集合 $N_{\mathcal{G}}$ 和弧(或有向边)集合 $A_{\mathcal{G}}$ 给出；每条弧形式为 $(u \rightarrow v)$ ，其中 $u, v \in N_{\mathcal{G}}$ 。 $\mathcal{G}$ 中的路径是端点相接的弧序列，例如从节点 u_1 到节点 u_k 的路径： $(u_1 \rightarrow u_2), (u_2 \rightarrow u_3), \dots, (u_{k-2} \rightarrow u_{k-1}), (u_{k-1} \rightarrow u_k)$ 。DAG (有向无环图) $\mathcal{G}$ 是没有环的有向图，即 DAG 中不存在上述形式且满足 $u_1 = u_k$ 的路径。

给定弧 $(u \rightarrow v) \in A_{\mathcal{G}}$ ， u 是 $\mathcal{G}$ 中 v 的父节点， v 是 u 的子节点。 $\mathcal{G}$ 中没有父节点的节点称为源节点，没有子节点的节点称为汇节点；其他节点为内部节点。

给定 DAG，反链是一个节点集合，集合中任意两个节点都“不可比较”，即对于任意两个不同节点 u 和 v ，既不存在从 u 到 v 的路径，也不存在从 v 到 u 的路径。反链的最大基数称为 DAG 的宽度。链是一条路径。如果 DAG 的每个节点都位于某条链上，则称一组链覆盖该 DAG (链可以“重叠”)。Dilworth 定理 [7] 指出，DAG 宽度等于覆盖它所需的最少链数。

一个计算用 DAG 建模，此时节点称为任务；为具体起见，假设 $N_{\mathcal{G}} = \{1, \dots, t\}$ ，并将集合 $\{1, \dots, t\}$ 记为 $[t]$ 。弧规定任务依赖：给定弧 $(u \rightarrow v)$ ，在 u 执行之前不能执行 v 。若一个任务集合中每个任务的全部父任务也都属于该集合，则称其满足优先关系约束。对于这样的集合 X ，用

$E(X)$ 表示不在 X 中、但其所有父任务均在 X 中的任务集合；当任务 X 已执行时，这些任务称为可执行任务。（因此，任何不在 X 中的源节点都可执行。）

任务执行用如下游戏建模。共有 n 名工作者，以 $[n]$ 的元素标识。令 X 是满足优先关系约束的任务集合。游戏从 $Y = X$ 开始，以轮次进行。每轮按照调度方案 Σ 将工作者分配到 $E(Y)$ 中的任务。方案指定分配 $\Sigma(Y)$ ，将每个工作者映射到集合 $E(Y) \cup \{\perp\}$ 的一个元素，即本轮某个可执行任务，或特殊元素 $\perp$ 。注意，分配由任务集合 Y 决定。可以将多名工作者分配给同一任务，也可以分配给不同任务，还允许工作者空闲。随后，每个已分配任务的工作者尝试执行该任务。分配给任务 j 的工作者 i ，以概率 $0 \leq p_{i,j} \leq 1$ 成功，并独立于其他尝试。假设对于每个任务，至少有一名工作者的成功概率非零。当且仅当分配给某任务的至少一名工作者成功时，该任务在本轮被执行。将所有已执行任务加入 Y ，再进入下一轮。也可能所有尝试都失败，此时集合 Y 不变，下一轮的工作者分配也保持不变。形式化地，调度方案 Σ 是函数 $\Sigma : 2^{N_G} \rightarrow ([n] \rightarrow (N_G \cup \{\perp\}))$ ；对于满足优先关系约束的任意任务子集 Z ， $\Sigma(Z)$ 是从 $[n]$ 到集合 $E(Z) \cup \{\perp\}$ 的函数。游戏持续进行，直到某轮中 G 的全部汇节点都属于 Y 。称游戏在该轮结束。

游戏的质量取决于结束速度。具体而言，在任务 X 已执行的初始条件下，从第一轮之后到游戏结束轮次的轮数，称为方案 Σ 的剩余完成时间。该时间是随机变量。如果 X 为空，则简称完成时间。目标是找到最小化期望完成时间的方案 Σ_{OPT} 。我们将该目标称为允许重算与超额分配的概率 DAG 调度问题（Recomputation and Overbooking allowed Probabilistic dAg Scheduling Problem, ROPAS）。

ROPAS

实例：描述 t 个任务依赖关系的 DAG G ，以及 n 名工作者。工作者 i 以概率 $0 \leq p_{i,j} \leq 1$ 成功执行任务 j ，并且对于任意任务 j ，存在工作者 i 满足 $p_{i,j} > 0$ 。

目标：找到最小化期望完成时间的方案 Σ_{OPT} 。

可以观察到，该优化问题的期望值有限。令 X 是满足优先关系约束的任务子集。 B_X 表示以任务 X 已执行为初始条件的全部方案中的最小期望剩余完成时间。下述引理指出 B_X 有限。证明虽简单，仍详细列出，以帮助读者熟悉记号。

引理 2.1. 对于满足优先关系约束的任意任务集合 X ， B_X 是有限的。

证明. 定义一个期望有限的方案，因此最小期望也必然有限。对 G 中由不属于 X 的任务诱导的子 DAG，取拓扑排序 $t_1, \dots, t_m$ 。可以按该排序执行任务，因为当 X 和 $t_1, \dots, t_{j-1}$ 已执行时，任务 t_j 可执行。我们的方案按此顺序，每次将所有工作者分配给当前任务。所有工作者都未能执行任务 t_j 的概率是 $\prod_{i=1}^n (1 - p_{i,t_j})$ ，根据假设严格小于 1。由于执行时间服从几何分布，该任务的期望执行时间为 $1/(1 - \prod_{i=1}^n (1 - p_{i,t_j}))$ 。利用期望线性性，方案的期望剩余完成时间就是排序中每个任务的期望之和。因此， B_X 至多为 $\sum_{j=1}^t 1/(1 - \prod_{i=1}^n (1 - p_{i,j}))$ ，得证。 $\square$

3 受限问题的调度算法

本节给出一个多项式时间算法，为 ROPAS 的受限版本寻找最优调度方案。具体作两个假设：(1) DAG G 的宽度至多为常数；(2) 工作者数量至多为常数。这两项限制看起来可能相当严格，甚至过于苛刻，但第 4 节将说明，它们从根本上是必要的。

我们构造一个有向图，建模计算如何演化。 G 的执行的可容许演化图 $\mathcal{A} = (N_{\mathcal{A}}, A_{\mathcal{A}})$ （示例见图 2）按归纳方式构造； $\mathcal{A}$ 的每个节点都是 G 的一个节点子集。从集合 $N_{\mathcal{A}} = \{\emptyset\}$ 开始。对不包含 G 全部汇节点的任意节点 $X \in N_{\mathcal{A}}$ ，计算 G 中的可执行任务集合 $E(X)$ 。然后，对每个非空子集 $D \subseteq E(X)$ ，若节点 $X \cup D$ 尚不在 $N_{\mathcal{A}}$ 中则加入；若弧 $(X, X \cup D)$ 尚不在 $A_{\mathcal{A}}$ 中则加入。由于 G 有限，该归纳过程显然定义唯一的有向图 $\mathcal{A}$ 。下面的引理说明其结构。

引理 3.1. 令 $\mathcal{A}$ 为 DAG $\mathcal{G}$ 的执行可容许演化图。则 $\mathcal{A}$ 是 DAG，其节点恰好是满足优先关系约束的任务集合。它只有一个源节点 $\emptyset$ 和一个汇节点 $N_{\mathcal{G}}$ 。

证明. 逐一验证各项断言。图 $\mathcal{A}$ 不可能有环，因为任意弧都从节点 X 指向节点 $X \cup D$ ，而 $X \cup D$ 的基数大于集合 X 。

$\mathcal{A}$ 的节点恰好是满足优先关系约束的任务集合。如果 X 满足优先关系约束，那么它与 $E(X)$ 任意子集的并集显然也满足；因此 $\mathcal{A}$ 的任意节点都满足约束。反过来，取任意满足约束的任务子集 Y ，令 $t_1, \dots, t_{|Y|}$ 为 $\mathcal{G}$ 中由 Y 诱导的子 DAG 的拓扑排序。对任意 j ，任务 t_j 显然在 $t_1, \dots, t_{j-1}$ 已执行时可执行。因此，如果 $\{t_1, \dots, t_{j-1}\}$ 是 $\mathcal{A}$ 的节点，则 $\{t_1, \dots, t_j\}$ 也是。由于 $\emptyset$ 是 $\mathcal{A}$ 的节点， Y 必然也是，由此也知 $N_{\mathcal{G}}$ 是 $\mathcal{A}$ 的节点。

只有存在从另一个节点指向 Y 的弧时，才将节点 Y 加入 $N_{\mathcal{A}}$ ，因此 Y 不是源节点。然而， $\emptyset$ 是源节点，因为不存在指向相同或更少元素集合的弧。

取 $\mathcal{A}$ 的任意节点 X ，假设它未包含 $\mathcal{G}$ 的全部汇节点。考察 $\mathcal{G}$ 的一个拓扑排序，可以发现：存在一个不在 X 中的 $\mathcal{G}$ 任务，其在 $\mathcal{G}$ 中的全部父任务都属于 X 。因此 $E(X)$ 非空， X 在 $\mathcal{A}$ 中有子节点。再取 $\mathcal{A}$ 中包含 $\mathcal{G}$ 全部汇节点的任意节点 X 。由于 X 满足优先关系约束，它包含全部任务，因此 $X = N_{\mathcal{G}}$ 。而 $E(N_{\mathcal{G}})$ 为空，所以 X 在 $\mathcal{A}$ 中没有子节点。 $\square$

给定调度方案 Σ ，可以将 $\mathcal{A}$ 转换为马尔可夫链，描述该方案下执行如何演化。具体而言，对于 $\mathcal{A}$ 的任意节点 X ， Σ 规定从 $E(X)$ 中选择任务并分配工作者，从而确定从 X 返回 X ，以及从 X 转移到 $\mathcal{A}$ 中子节点的概率。期望剩余完成时间就是到达 $\mathcal{A}$ 汇节点的期望首次到达时间。利用马尔可夫链理论，可以得到以下 $\mathcal{A}$ 各节点期望剩余完成时间之间的关系。

定理 3.2 ([29]). 考虑方案 Σ 和满足优先关系约束、但未包含 $\mathcal{G}$ 全部汇节点的任务集合 X 。令 $D_0, D_1, \dots, D_k$ 是 $E(X)$ 的互不相同子集，并令 $D_0 = \emptyset$ 。令 a_i 表示在分配 $\Sigma(X)$ 下，工作者恰好执行任务集合 D_i 的概率。令 $X_i = X \cup D_i$ 。则

$$T_{X_0} = \frac{1}{a_1 + \dots + a_k} \left(1 + \sum_{i=1}^k a_i \cdot T_{X_i} \right),$$

其中 T_{X_i} 是任务 X_i 已执行时，方案 Σ 的期望剩余完成时间；约定 $1/0 = \infty$ 、 $0 \cdot \infty = 0$ 。

但目标是寻找最小化期望的方案，而非只计算给定方案的期望完成时间。为此，我们给出动态规划算法 OPT，构造方案 Σ_{OPT} 。由于 $\mathcal{A}$ 无环，可以从汇节点开始向源节点回溯，应用定理 3.2 的递推式。具体而言，先任意初始化方案，再取 $\mathcal{A}$ 的拓扑排序 $Y_1, \dots, Y_m$ ，按逆序处理。伪代码见图 1。处理 $\mathcal{A}$ 的节点 X 时，确定两个值：数值 S_X 和分配 $\Sigma_{OPT}(X)$ 。首先将 S_{Y_m} 设为 0，并定义 $\Sigma_{OPT}(Y_m)$ ，使每名工作者都被分配到 $\perp$ 。现在令 $1 \leq h < m$ ，说明如何定义 S_{Y_h} 和 $\Sigma_{OPT}(Y_h)$ 。令 $D_0, \dots, D_k$ 是 $E(Y_h)$ 的互不相同子集，其中 $D_0 = \emptyset$ 。考虑将 n 名工作者分配到 $E(Y_h)$ 任务、但不分配到 $\perp$ 的全部 $|E(Y_h)|^n$ 种分配。对每种分配，计算工作者恰好执行任务集合 D_i 的概率 a_i 。若 $a_1 + \dots + a_k > 0$ ，则计算加权和 $1/(a_1 + \dots + a_k) \cdot (1 + \sum_{i=1}^k a_i S_{Y_h \cup D_i})$ ，选择最小化该和的分配。将 S_{Y_h} 设为该最小值，并将 $\Sigma_{OPT}(Y_h)$ 设为达到最小值的分配。随后递减 h ，在拓扑排序中后退，处理另一个节点。处理完全部排序后，就确定了方案 Σ_{OPT} 。

我们的目标有两个。首先，证明方案 Σ_{OPT} 确实最小化期望完成时间。为此，考虑允许工作者空闲的扩展算法 OPT*。这种自由度与 ROPAS 定义一致，可通过递推式证明 OPT* 求解 ROPAS。然后证明没有必要空闲，因为始终将每名工作者分配到任务的方案中也能找到最优解。因此，OPT 也能求解 ROPSA。最后证明，当 DAG 宽度和工作者数量至多为常数时，OPT 在多项式时间内运行。

为便于分析，考虑允许工作者空闲的扩展算法 OPT*。处理 Y_h 时，考虑将 n 名工作者分配到 $E(Y_h)$ 中的任务或 $\perp$ 的全部 $(|E(Y_h)| + 1)^n$ 种分配。处理拓扑排序后得到的方案记为 Σ_{OPT^*} 。下

数据结构: T 是将 $\mathcal{A}$ 的节点映射到互不相同浮点变量的字典

$OPT(t, \mathcal{G}, n, (p_{i,j}))$	08	for 全部非空子集 $D \subseteq I$
01 let $Y_1, \dots, Y_m$ 为 $\mathcal{A}$ 的一个拓扑排序	09	let $a = \Pr[A \text{ 恰好执行 } D]$
02 $S_{Y_m} = 0$	10	$sum = sum + a \cdot S_{Y_h \cup D}$
03 for $h = m - 1$ downto 1 do	11	let $a = \Pr[\text{所有已分配工作者都失败}]$
04 $min = \infty$	12	if $(1 + sum)/(1 - a) < min$
05 for 将工作者分配到 $E(Y_h)$ 的全部分配 A	13	$min = (1 + sum)/(1 - a)$
06 let $I \subseteq E(Y_h)$ 为被分配的任务集合	14	$\Sigma_{OPT}(Y_h) = A$
07 $sum = 0$	15	$S_{Y_h} = min$

图 1: 构造最优调度方案 Σ_{OPT} 的算法: DAG $\mathcal{G}$ 描述 t 个任务的依赖关系, 另有 n 名工作者, 其中工作者 i 以概率 $p_{i,j}$ 成功执行任务 j 。

述命题解释为何扩展算法找到最佳方案。(注意这里量词顺序很“强”: 一个单一方案, 在 $\mathcal{A}$ 的所有 X 上, 与从 X 开始的任意方案比较时均最优。)

命题 3.3. 当算法 OPT^* 终止并求出方案 Σ_{OPT^*} 后, 对 $\mathcal{A}$ 的任意节点 X , 以任务 X 已执行为初始条件时, 该方案的期望剩余完成时间等于任意方案从任务 X 已执行状态开始所能达到的最小期望剩余完成时间 B_X 。

证明. 令 $Y_1, \dots, Y_m$ 为算法 OPT^* 使用的 $\mathcal{A}$ 的拓扑排序。下面证明, 在逆序处理过程中, 以下以 h 为参数的不变式成立:

对全部满足 $h \leq i \leq m$ 的 i , 有 $S_{Y_i} = B_{Y_i}$, 且 S_{Y_i} 等于任务 Y_i 已执行时, Σ_{OPT^*} 的期望剩余完成时间。

当 $h = m$ 时, 不变式显然成立。根据引理 3.1, $\mathcal{A}$ 只有一个汇节点 $N_{\mathcal{G}}$, 因此 $Y_m = N_{\mathcal{G}}$ 。如果全部任务已执行, 最小期望剩余完成时间为零。因此, 算法将 S_{Y_m} 设为零是正确的。任何从全部任务已执行状态开始的方案, 其期望都为零, 所以 $\Sigma_{OPT^*}(Y_m)$ 的值满足要求。

任取 $2 \leq h \leq m$ 。我们将说明, OPT^* 处理 Y_{h-1} 后, 将 h 减一时不变式成立。令 $X = Y_{h-1}$ 。由于 X 不是 $\mathcal{A}$ 的汇节点, 至少有一个子节点。令其子节点为 $X_1, \dots, X_k$ 。

首先证明, 赋给 S_X 的值至多为最小期望剩余完成时间 B_X 。选取一个在任务 X 已执行时达到最小值的方案 Σ 。 $\Sigma(X)$ 可能将部分工作者分配给 $\perp$ 。引理 2.1 保证 B_X 有限。利用定理 3.2, 可将该方案的期望表示为如下加权和:

$$B_X = 1/(a_1 + \dots + a_k) \cdot (1 + \sum_{i=1}^k a_i T_{X_i})$$

其中 T_{X_i} 是任务集合 X_i 已执行时 Σ 的期望剩余完成时间。由不变式 $S_{X_i} = B_{X_i}$, 期望 T_{X_i} 至少为 S_{X_i} 。动态规划将每种分配都作为 $\Sigma_{OPT^*}(X)$ 的候选, 特别也会考虑 $\Sigma(X)$ 。对该分配, 算法计算加权和 $1/(a_1 + \dots + a_k) \cdot (1 + \sum_{i=1}^k a_i S_{X_i})$, 因此该和至多为 B_X 。算法选择最小化加权和的分配, 所以 S_X 的值至多为 B_X , 得证。

选定 S_X 和 $\Sigma_{OPT^*}(X)$ 后, S_X 显然等于 Σ_{OPT^*} 从 X 已执行状态开始的期望。因为对算法考虑的任意分配, 由定理 3.2, 该加权和等于: 以任务 X 已执行为初始条件, 并以该分配替代 $\Sigma_{OPT^*}(X)$ 时, Σ_{OPT^*} 的期望剩余完成时间。

由于 Σ_{OPT^*} 的期望为 S_X , 而 S_X 至多为 B_X , 所以 Σ_{OPT^*} 从 X 已执行状态开始的期望剩余完成时间实际上等于 B_X 。因此, 将 h 减一后, 不变式仍成立。证明完成。□

利用这一结果，我们将说明工作者没有必要空闲。首先观察到，粗略地说，“完成得越多，剩余完成时间越少”，下述引理给出详细表述。

引理 3.4. 对于满足优先关系约束的任意任务子集 $X \subseteq Y$ ，有 $B_X \geq B_Y$ 。

证明. 只需在对 Y 附加一个限制的情况下证明引理。具体而言，对所述任意 X 、任意 $u \in E(X)$ ，令 $Y = X \cup \{u\}$ 。按 X 的基数逆向归纳。 $|X| = t - 1$ 时结论显然成立，因为此时 $B_Y = 0$ 。

在归纳步骤中，取基数至多为 $t - 2$ 的 X 。考虑方案 Σ_{OPT^*} 。根据命题 3.3，对于满足优先关系约束的任意 Z ， Σ_{OPT^*} 从任务 Z 已执行状态开始的期望剩余完成时间是 B_Z 。令 $t_1, \dots, t_k$ 是 $\Sigma_{OPT^*}(X)$ 分配工作者的任务。由于 B_X 有限， $k \geq 1$ 。令 p_i 表示分配到任务 t_i 的工作者中至少一名成功的概率。令 $p_C = \prod_{i \in C} p_i \prod_{i \in [k] \setminus C} (1 - p_i)$ 表示在 $\Sigma_{OPT^*}(X)$ 中，索引属于 $[k]$ 的任务中恰好执行索引集合 C 的任务的概率。利用定理 3.2，可以将 Σ_{OPT^*} 的期望剩余完成时间表示为

$$B_X = \frac{1 + \sum_{\emptyset \subset C \subseteq [k]} p_C \cdot B_{X \cup C}}{1 - p_\emptyset} \quad (1)$$

其中 $p_\emptyset < 1$ 。

首先分析几个较简单的特殊情况，它们也会简化后续分析。先假设 $\Sigma_{OPT^*}(X)$ 没有将任何工作者分配到 u 。此时，任务 $X \cup \{u\}$ 已执行后，每个已分配的任务仍可执行。因此，可以定义新方案 Σ' ，除 $\Sigma'(X \cup \{u\}) = \Sigma_{OPT^*}(X)$ 外，与 Σ_{OPT^*} 完全相同。于是， Σ' 从任务 $X \cup \{u\}$ 已执行状态开始的期望剩余完成时间为

$$T'_{X \cup \{u\}} = \frac{1 + \sum_{\emptyset \subset C \subseteq [k]} p_C \cdot B_{X \cup C \cup \{u\}}}{1 - p_\emptyset}.$$

根据归纳假设，显然有 $B_{X \cup C} \geq B_{X \cup C \cup \{u\}}$ ，因此 $B_X \geq T'_{X \cup \{u\}}$ 。又因为 $T'_{X \cup \{u\}} \geq B_{X \cup \{u\}}$ ，得到 $B_X \geq B_{X \cup \{u\}}$ ，得证。因此，下文可假设 $t_k = u$ 。再假设 $k = 1$ 。此时， B_X 等于 $1/(1 - p_\emptyset)$ 加 $B_{X \cup \{t_k\}}$ （因为 $p_\emptyset$ 不可能为 1），所以后者显然小于 B_X 。当 $k \geq 2$ 、但 $0 = p_1 = \dots = p_{k-1}$ 时，也得到类似结论。

处理完较简单情况后，考虑最后且最有意义的情形： $k \geq 2$ 、 $t_k = u$ ，且 $p_1, \dots, p_{k-1}$ 中至少一个严格大于 0。 B_X 的式 (1) 包含事件概率，可将事件分为三组：

- (i) 只有任务 t_k 执行，概率为 $p_k \cdot (1 - p_1) \cdot \dots \cdot (1 - p_{k-1})$ ；
- (ii) 恰好执行非空集合 $C \subseteq [k-1]$ 索引的任务，同时执行任务 t_k ，概率为 $p_k \cdot \prod_{i \in C} p_i \prod_{i \in [k-1] \setminus C} (1 - p_i)$ ；
- (iii) 恰好执行非空集合 $C \subseteq [k-1]$ 索引的任务，但不执行任务 t_k ，概率为 $(1 - p_k) \cdot \prod_{i \in C} p_i \prod_{i \in [k-1] \setminus C} (1 - p_i)$ 。

B_X 的式 (1) 中，第二组事件的概率乘以 $B_{X \cup C \cup \{t_k\}}$ ，第三组事件的概率乘以 $B_{X \cup C}$ 。由于 C 非空，根据归纳假设 $B_{X \cup C \cup \{t_k\}} \leq B_{X \cup C}$ 。因此可以将后者替换为前者，所得表达式不会增大。随后将第二组的每项与第三组对应项逐一合并，得到

$$B_X \geq \frac{1 + p_k q_\emptyset \cdot B_{X \cup \{t_k\}} + \sum_{\emptyset \subset C \subseteq [k-1]} q_C \cdot B_{X \cup C \cup \{t_k\}}}{1 - (1 - p_k) q_\emptyset},$$

其中 $q_C = \prod_{i \in C} p_i \prod_{i \in [k-1] \setminus C} (1 - p_i)$ 表示 $[k-1]$ 中恰好执行索引集合 C 对应任务的概率。用 S 表示其中的大求和式。现在只需证明

$$\frac{1 + p_k q_\emptyset \cdot B_{X \cup \{t_k\}} + S}{1 - (1 - p_k) q_\emptyset} \geq B_{X \cup \{t_k\}}.$$

为验证该不等式，将两边乘以分母，再按 $B_{X \cup \{t_k\}}$ 合并，转化为等价形式：

$$\frac{1+S}{1-q_0} \geq B_{X \cup \{t_k\}},$$

因为 $q_0 < 1$ 。左侧可视为方案 Σ'' 从任务 $X \cup \{t_k\}$ 已执行状态开始的期望剩余完成时间。 Σ'' 除如下情形外与 Σ_{OPT^*} 相同：当需要执行任务 $X \cup \{t_k\}$ 时，按照 $\Sigma_{OPT^*}(X)$ 的方式分配工作者，但省略原分配给 t_k 的工作者，让这些工作者不被分配。由于 $B_{X \cup \{t_k\}}$ 是最小期望，必然小于或等于该方案 Σ'' 的期望。

归纳步骤及证明完成。 $\square$

利用引理可以说明：处理 Y_h 时，只需考虑将每名工作者映射到一个可执行任务的分配 S ，无需将任何工作者分配到 $\perp$ 。

命题 3.5. 对于满足优先关系约束、但未包含 $\mathcal{G}$ 全部汇节点的任意任务集合 X ，存在方案 Σ ，使 $\Sigma(X)$ 将每名工作者分配给 $E(X)$ 中的任务，且任务 X 已执行时， Σ 的期望剩余完成时间为 B_X (即最小值)。

证明. 利用命题 3.3 的方案 Σ_{OPT^*} 构造方案 Σ 。我们将证明 Σ 的期望至多为 Σ_{OPT^*} 的期望；后者已是最小值，因此两者期望必然相等。

首先观察 Σ_{OPT^*} 。令 $t_1, \dots, t_k$ 为 $\Sigma_{OPT^*}(X)$ 分配工作者的任务，已知 $k \geq 1$ 。若 $\Sigma_{OPT^*}(X)$ 已将每名工作者分配到任务，结论立即成立。因此，假设至少有一名工作者未被 $\Sigma_{OPT^*}(X)$ 分配到任何任务。

构造方案 Σ 。令 U 为未被 $\Sigma_{OPT^*}(X)$ 分配到任务的工作者集合。定义 Σ ，除分配 $\Sigma(X)$ 还将 U 中全部工作者分配给任务 t_1 外，与 Σ_{OPT^*} 相同。令 q 为 U 中工作者被分配到 t_1 时至少一名成功的概率， $0 \leq q \leq 1$ 。下面证明，任务 X 已执行时 Σ 的期望剩余完成时间 T_X 至多为 B_X 。

比较 B_X 和 T_X 的等式。与此前相同，令 p_C 表示恰好执行由 C 索引的任务的概率。根据定理 3.2， B_X 可表示为

$$B_X = \frac{1 + \sum_{\emptyset \subset C \subseteq [k]} p_C \cdot B_{X \cup C}}{1 - p_\emptyset},$$

并且已知 $p_\emptyset < 1$ 。类似地，可表示 T_X 。对任意满足 $\emptyset \subset C \subseteq [k]$ 的 C ，在 $\Sigma_{OPT^*}(X)$ 中恰好执行 C 中任务的事件，可进一步分解成两个互斥“子事件”： U 中至少一名工作者成功，或 U 中全部工作者失败。因此，

$$T_X = \frac{1 + q \cdot p_\emptyset \cdot B_{X \cup \{t_1\}} + \sum_{\emptyset \subset C \subseteq [k]} q \cdot p_C \cdot B_{X \cup C \cup \{t_1\}} + \sum_{\emptyset \subset C \subseteq [k]} (1 - q) \cdot p_C \cdot B_{X \cup C}}{1 - (1 - q) \cdot p_\emptyset},$$

即使对某些 C 有 $t_1 \in C$ ，该式也成立。

作两个简化假设。若 $q = 0$ ，则 U 中工作者始终失败， T_X 显然等于 B_X 。若 $p_\emptyset = 0$ ，则必有一个任务确定执行；此时可将 U 中工作者改分配到该任务，而非 t_1 ，所得 T_X 仍等于 B_X 。因此，不失一般性，可假设 $q > 0$ 且 $p_\emptyset > 0$ 。

修改 T_X 的表达式，使其更接近 B_X 。由引理 3.4，有 $B_{X \cup C \cup \{t_1\}} \leq B_{X \cup C}$ 。因此，在 T_X 表达式中将前者替换为后者，并逐项合并，可得到上界：

$$T_X \leq \frac{1 + q \cdot p_\emptyset \cdot B_{X \cup \{t_1\}} + \sum_{\emptyset \subset C \subseteq [k]} p_C \cdot B_{X \cup C}}{1 - (1 - q) \cdot p_\emptyset}, \quad (2)$$

它开始接近 B_X 的表达式。

下面证明式 (2) 的上界至多为 B_X ，从而得到 $T_X \leq B_X$ ，也就有 $T_X = B_X$ 。令 S 为分子中的求和式。目标可重述为证明

$$\frac{1 + q \cdot p_\emptyset \cdot B_{X \cup \{t_1\}} + S}{1 - (1 - q) \cdot p_\emptyset} \leq B_X = \frac{1 + S}{1 - p_\emptyset}.$$

该不等式可等价表示为

$$B_{X \cup \{t_1\}} \leq \frac{B_X (1 - p_\emptyset + q \cdot p_\emptyset) - 1 - S}{q \cdot p_\emptyset} = \frac{1 + S}{1 - p_\emptyset} \left(\frac{1 - p_\emptyset}{q \cdot p_\emptyset} + 1 \right) - \frac{1 + S}{q \cdot p_\emptyset} = \frac{1 + S}{1 - p_\emptyset} = B_X,$$

因为 $q, p_\emptyset > 0$ 。由引理 3.4, $B_{X \cup \{t_1\}}$ 确实至多为 B_X ，因此证明完成。 $\square$

推论 3.6. 算法 OPT 找到最小化期望完成时间的方案 Σ_{OPT} 。

下面处理第二个目标：证明当 DAG 宽度至多为常数，且工作者数量至多为常数时，算法在多项式时间内运行。令 w 为 $\mathcal{G}$ 的宽度。根据 Dilworth 定理，DAG $\mathcal{G}$ 可由 w 条链覆盖。如果链中任意任务已执行，该链中所有更早任务也必须已执行。每条链长度至多为 t ，因此满足优先关系约束的不同任务子集至多有 $(t + 1)^w$ 个。算法处理 $\mathcal{A}$ 的节点时，考虑将工作者分配到可执行任务。每名工作者至多有 t 种分配方式，所以分配总数至多为 t^n 。对于给定分配，实际上只需对至多 2^n 个集合 D_i 计算概率 a_i ：因为工作者最多分配到 n 个不同任务，不属于这些任务子集的集合，其对应概率必为零，可从加权和省略。由此得到本节主要结果。

定理 3.7. 当 DAG 宽度受某个常数限制，且工作者数量也受某个常数限制时，算法 OPT 在多项式时间内求解 ROPAS 问题。

算法在一个 DAG 上的应用示例见图 2。

4 调度的复杂度

本节说明上一节算法之所以受到如此严格限制的根本原因。回顾一下，该算法将工作者数量和 DAG 宽度都限制为至多常数。事实证明，如果允许工作者数量增长、而 DAG 宽度至多为常数，或允许 DAG 宽度增长、而工作者数量至多为常数，ROPAS 都是 NP 难的。因此，这个问题的多项式时间算法必须同时限制 DAG 宽度和工作者数量，除非 $P \neq NP$ 。我们还证明，如果同时允许工作者数量和 DAG 宽度增长，那么除非 $P=NP$ ，否则该问题无法在小于 $5/4$ 的近似因子内得到近似解。

我们的归约表明，ROPAS 问题的复杂度会非常陡峭地上升。首先，当 DAG 只有一个任务时，问题是平凡的：最优调度方案只需把每名工作者都分配给该任务。然而，当 DAG 只有两个独立任务时，问题就变成 NP 难的！（可以认为，这是“第二简单”的 DAG。）直观而言，这种困难来自在可执行任务之间完美平衡不可靠工作者负载的难度。其次，当只有一名工作者时，问题很容易：最优调度方案只需遵循任意一种拓扑排序，并把这名唯一的工作者分配给当前任务。然而，当只有两名工作者时，问题就变成 NP 难的！直观而言，此时的困难来自可能十分复杂的任务依赖：很难确定应该选择哪些任务执行，才能使足够多的任务变为可执行，从而避免工作者被迫空闲。

4.1 使用大量工作者调度窄 DAG 是 NP 难的

本节证明，当 DAG 被限制为两个独立任务、但工作者数量可以增长时，ROPAS 是 NP 难的。为此，我们引入两个辅助问题并确定它们的复杂度。第一个是乘法划分问题。

乘法划分 (Multiplicative Partition, MP)

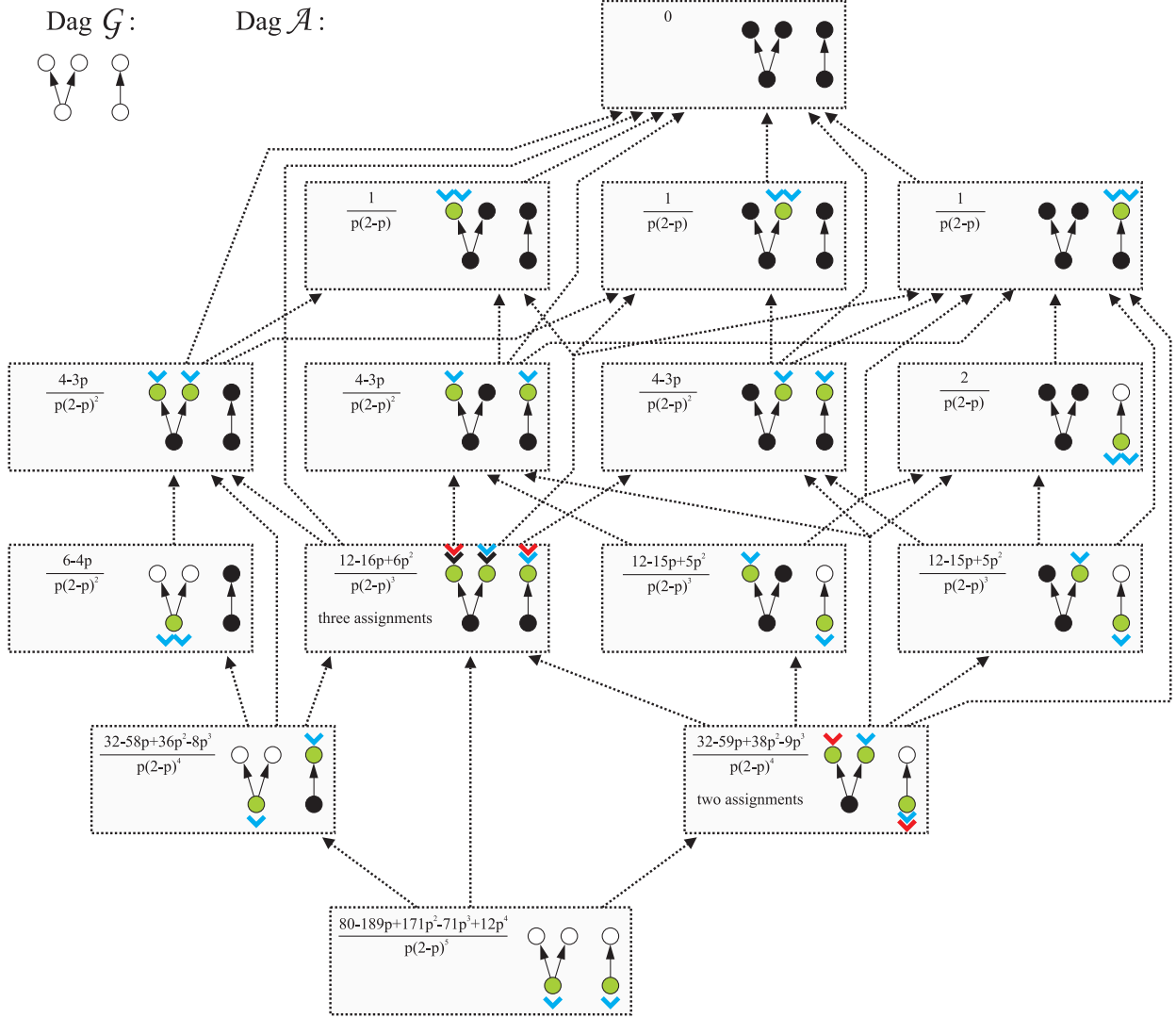


图 2: 图中展示了为给定 DAG $\mathcal{G}$ 构造的执行可容许演化 DAG $\mathcal{A}$ 。虚线框与虚线箭头表示 $\mathcal{A}$ 的节点和弧。还展示了两个工作者情形的最优方案：每名工作者对任意任务都以相同概率 $0 < p < 1$ 成功执行。每个框包含： $\mathcal{G}$ 中已执行任务的子集（黑色）及此时可执行任务（绿色）；最小期望剩余完成时间（左上角）；以及达到该期望所需的工作者分配方式（同色“v”符号）。图中展示了达到该期望的全部分配，省略交换两名工作者所得的对称分配。注意，第一轮必须将两名工作者分配给不同任务，否则无法最小化期望完成时间。

实例：数字 n ，整数大小 $s_1, \dots, s_n \geq 2$ 。

问题：是否存在集合 P ，使得 $\prod_{i \in P} s_i = \prod_{i \notin P} s_i$ ？

很容易想到从著名的划分问题归约，以证明 MP 的 NP 完全性。这里可以尝试利用如下事实： $\sum_{i \in P} r_i = \sum_{i \notin P} r_i$ 当且仅当 $\prod_{i \in P} 2^{r_i} = \prod_{i \notin P} 2^{r_i}$ 。遗憾的是，表示 $s_i = 2^{r_i}$ 所需空间相对于表示 r_i 所需空间是指数级的。通过从三元集合精确覆盖（Exact Cover by 3-Sets, X3C）问题作出更精细的归约，并使用数论的一些基本事实，可以避免这种空间爆炸。

引理 4.1. *MP 问题是 NP 完全的。*

证明. 我们将给出从三元集合精确覆盖问题到乘法划分问题的归约。

取 X3C 的任意一个实例：由集合 $U = \{1, \dots, 3m\}$ 的若干个 3 元子集构成的集合族 $C = \{S_1, \dots, S_n\}$ ，其中 $S_i = \{a_i, b_i, c_i\}$ 。回顾一下，当且仅当存在 C 的一个子集合族，使得 U 的每个元素都恰好出现在其中一个子集中时，该实例才是肯定实例。我们可以假设 $m \leq n$ ，且 C 中所有子集的并集是 U ；否则，该 X3C 实例显然是否定实例。按如下方式构造 MP 实例。首先枚举连续的前若干个素数 $p_1, \dots, p_{3m+1}$ 。由素数定理（例如参见 [17]），素数 p_{3m+1} 的值为 $O(m \ln m)$ 。因此，可以用 Eratosthenes 筛法在 $O(m^2 \log^2 m)$ 时间内枚举这些素数；相对于 X3C 实例大小，该时间是多项式的。MP 问题的实例包含 $n+2$ 个大小。前 n 个被定义为以子集元素为下标的素数之积，即 $s_i = p_{a_i} \cdot p_{b_i} \cdot p_{c_i} \geq 2$ 。注意，各个 s_i 都相当小，每个 s_i 都是 $O(n^6)$ 。我们增加两个“辅助”大小。令 p 为从 s_1 到 s_n 的乘积， q 为前 $3m$ 个素数的乘积；两者都是 $O(n^{6n})$ ，因而足够小。注意，因为 $\bigcup_i S_i = U$ ，所以 p/q 是整数。定义 $s_{n+1} = p_{3m+1} \cdot p/q$ 和 $s_{n+2} = p_{3m+1} \cdot q$ 。因此，可以在关于 n 的多项式时间和空间内表示该 MP 实例，满足我们的要求。

假设 X3C 实例是肯定实例，并令 Q 为定义该子集合族的下标集合。那么， $\prod_{i \in Q} s_i$ 就是从 p_1 到 p_{3m} 的所有素数的乘积。因此， $\prod_{i \notin Q} s_i$ 等于所有 s_i 的乘积除以这些素数的乘积。于是，

$$s_{n+1} \prod_{i \in Q} s_i = s_{n+2} \prod_{i \notin Q} s_i ,$$

所以 MP 实例是肯定实例。

假设 MP 实例是肯定实例，并令 P 为满足 $\prod_{i \in P} s_i = \prod_{i \notin P} s_i$ 的子集。注意，只有 s_{n+1} 和 s_{n+2} 这两个大小含有素因子 p_{3m+1} 。两个自然数相等，当且仅当它们具有相同的素因子分解。因此，要使两个乘积相等， $n+1$ 和 $n+2$ 中必须恰有一个属于 P 。不失一般性，可以假设 $n+1$ 属于 P 。令 $L = P \setminus \{n+1\}$ ，并令 R 包含所有不属于 P 且不是 $n+2$ 的元素。于是，

$$s_{n+1} \cdot \prod_{i \in L} s_i = s_{n+2} \cdot \prod_{i \in R} s_i ,$$

等价地，可以将其写为

$$p/q \cdot \prod_{i \in L} s_i = q \cdot \prod_{i \in R} s_i .$$

当然， L 和 R 划分 $\{1, \dots, n\}$ ，所以 $p = \prod_{i \in L} s_i \cdot \prod_{i \in R} s_i$ 。因此， $q^2 = (\prod_{i \in L} s_i)^2$ 。由于这些数都是正数， $q = \prod_{i \in L} s_i$ 。于是，存在若干个 s_i ，它们的乘积恰好包含从 p_1 到 p_{3m} 的每个素数一次。这个集合 L 定义了 C 的一个子集合族，使得 U 的每个元素都恰好包含在该子集合族的一个子集中。因此，X3C 实例是肯定实例。 □

我们利用 MP 问题，证明第二个辅助问题的 NP 难性；这个问题称为乘积之和最小划分。

乘积之和最小划分 (Partition with Minimum Sum of Products, PMSP)

实例：数字 n ，有理数 $0 < r_1, \dots, r_n < 1$ 。

目标：找到使下式最小的集合 P ： $\prod_{i \in P} r_i + \prod_{i \notin P} r_i$ 。

引理 4.2. *PMSP 问题是 NP 难的。*

证明. 我们给出从 MP 到 PMSP 的多项式时间 Turing 归约。具体而言, 对于给定的 MP 实例, 我们构造一个适当的 PMSP 实例, 再检查其最小值, 从而找到 MP 实例的答案。证明基于如下事实: 对任意 $d > 0$, 定义在 $0 \leq x \leq d$ 上的函数 $f(x) = e^x + e^{d-x}$ 恰好在 $x = d/2$ 时取得最小值。困难在于确保计算过程只使用有理数。

取 MP 问题的任意实例, 并令 $s_1, \dots, s_n$ 为其中的正整数大小, 每个大小至少为 2。通过取 $r_i = h_i^2$ 定义一个 PMSP 实例, 其中 $h_i = 1/s_i$ 。于是 $0 < r_i < 1$, 满足要求。下面将说明, 只需检查该 PMSP 实例的最小值是否等于 $2 \prod_{i=1}^n h_i$, 就能准确判断 MP 实例是否为肯定实例。

考察乘积表达式 $\prod_{i \in P} r_i + \prod_{i \notin P} r_i$ 何时达到该值。令 $d = \sum_{i=1}^n \ln r_i$ 。于是,

$$\prod_{i \in P} r_i + \prod_{i \notin P} r_i = e^{\sum_{i \in P} \ln r_i} + e^{d - \sum_{i \in P} \ln r_i} \geq 2e^{d/2}$$

仅当 $d/2 = \sum_{i \in P} \ln r_i$ 时等号成立。关键观察是, $2e^{d/2}$ 就是 $2 \prod_{i=1}^n h_i$, 而它是有理数。因此, 可以调用一个求解 PMSP 的预言机来处理我们构造的实例, 检查其返回的最小值, 判断是否存在 P 满足 $d/2 = \sum_{i \in P} 2 \ln h_i$ 。这个等式又可以等价地写为 $1/2 \sum_{i=1}^n \ln h_i = \sum_{i \in P} \ln h_i$, 这等价于 $\sum_{i \in P} \ln h_i = \sum_{i \notin P} \ln h_i$, 进一步等价于 $\prod_{i \in P} h_i = \prod_{i \notin P} h_i$ 。而这又等价于 $\prod_{i \in P} s_i = \prod_{i \notin P} s_i$ 。因此, MP 实例为肯定实例, 当且仅当 PMSP 实例的最小值等于 $2 \prod_{i=1}^n h_i$ 。 $\square$

辅助工具已经准备完毕, 现在可以证明本小节的主要困难性结论。

定理 4.3. 当 DAG 被限制为两个独立任务 (而工作者数量可以增长) 时, ROPAS 问题是 NP 难的。

证明. 我们给出从 PMSP 到 ROPAS 的多项式时间 Turing 归约。取任意 $0 < r_1, \dots, r_n < 1$, 考虑一个仅含两个孤立节点的 DAG, 分别称它们为左任务和右任务。共有 n 名工作者。将工作者 i 的成功概率 $p_{i, \text{left}}$ 和 $p_{i, \text{right}}$ 都定义为 $1 - r_i$, 并简记为 p_i 。选取使期望完成时间最小的调度方案 Σ 。令 L 为 $\Sigma(\emptyset)$ 分配给左任务的工作者集合, 即两个任务都可执行时分配给左任务的工作者。我们将说明, 期望完成时间最小, 当且仅当 $\prod_{i \in L} r_i + \prod_{i \notin L} r_i$ 最小。因此, 可以把最优调度方案转换为该 PMSP 实例的解。

为说明这一等价关系, 我们依次分析调度方案必须作出的分配。令 R 为 $\Sigma(\emptyset)$ 分配给右任务的工作者集合。因此, L 和 R 互不相交, 且都包含于 $\{1, \dots, n\}$ 。令 $\ell = 1 - \prod_{i \in L} (1 - p_i)$ 表示左侧至少一名工作者成功的概率, $r = 1 - \prod_{i \in R} (1 - p_i)$ 表示右侧至少一名工作者成功的概率。可以用定理 3.2 计算期望完成时间 $T_\emptyset$:

$$T_\emptyset = \frac{1 + \ell(1 - r) \cdot T_{\{\text{left}\}} + (1 - \ell)r \cdot T_{\{\text{right}\}}}{1 - (1 - \ell)(1 - r)},$$

其中, $T_{\{\text{left}\}}$ 是左任务已经执行完成时, 该调度方案完成剩余任务的期望时间; $T_{\{\text{right}\}}$ 是右任务已经执行完成时的对应期望时间。

观察到 $T_{\{\text{left}\}}$ 与 $T_{\{\text{right}\}}$ 相等后, 可以简化这个表达式。实际上, 将集合 L 和 R 视为常量时, $T_\emptyset$ 在 $T_{\{\text{left}\}}$ 和 $T_{\{\text{right}\}}$ 都最小时取得最小值。令 $K \subseteq \{1, \dots, n\}$ 为 $\Sigma(\{\text{right}\})$ 分配给左任务的工作者集合, 即右任务已经执行完成、左任务是唯一可执行任务时的分配。那么, 期望剩余完成时间 $T_{\{\text{right}\}}$ 为 $1 / (1 - \prod_{i \in K} (1 - p_i))$ 。因为 $0 < p_i < 1$, 显然该期望最小, 当且仅当 $K = \{1, \dots, n\}$ 。利用同样的论证可知, 当右任务是唯一尚未执行完成的任务时, 该方案也会把所有工作者都分配给它。因此, $T_{\{\text{left}\}} = T_{\{\text{right}\}}$ 。将这个期望记为 T 。注意 $T > 0$ 。于是,

$$T_\emptyset = \frac{1 + T(\ell(1 - r) + (1 - \ell)r)}{1 - (1 - \ell)(1 - r)} = T + \frac{1 - T\ell r}{\ell + r - \ell r}.$$

我们观察到, $\Sigma(\emptyset)$ 会把每名工作者都分配给某个任务。为证明这一点, 我们说明该期望会随 ℓ 或 r 增大而减小。注意该公式关于 ℓ 和 r 对称, 因此只计算它对 r 的偏导数:

$$\frac{\partial}{\partial r} = \frac{-1 + \ell - T\ell^2}{(\ell + r - \ell r)^2}.$$

对于任意 $0 < \ell < 1$, 这个导数都严格为负。因此, 当 r 和 ℓ 最大时, 期望最小。这等价于每名工作者都必须得到分配; 换言之, 集合 L 和 R 划分集合 $\{1, \dots, n\}$ 。

概括目前的观察: 调度方案最初会把每名工作者都分配给某个任务; 只剩一个任务尚未执行完成时, 会把所有工作者都分配给该任务。

利用这些观察, 我们回到 $T_\emptyset$ 在何种条件下最小的问题。因为 L 和 R 划分 $\{1, \dots, n\}$, 所以 $\ell + r - \ell r$ 恰好是 n 名工作者中至少一名成功的概率, 其值与工作者在左右任务之间的分配方式无关。此时 T 也是常量, 因此该期望最小, 当且仅当在 $\{1, \dots, n\}$ 的各种划分 L 和 R 中, ℓr 最大。而

$$\begin{aligned} \ell r = 1 - & \left(\prod_{i \in L} (1 - p_i) \right) - \left(\prod_{i \in R} (1 - p_i) \right) + \\ & \left(\prod_{i \in L} (1 - p_i) \prod_{i \in R} (1 - p_i) \right), \end{aligned}$$

由于 L 和 R 构成划分, 第三个大括号中的表达式与 L 和 R 的选择无关。因此, 对于使下式最小的划分 L 和 R , 上述表达式最大: $\prod_{i \in L} r_i + \prod_{i \in R} r_i$ 。

概括以上观察: 调度方案使期望最小, 当且仅当其集合 L 使 $\prod_{i \in L} r_i + \prod_{i \notin L} r_i$ 最小, 且集合 $K = \{1, \dots, n\}$ 。定理由此得证。 $\square$

4.2 使用少量工作者调度宽 DAG 是 NP 难的

我们证明, 当工作者被限制为两名、但 DAG 宽度可以增长时, ROPAS 问题是 NP 难的。为此, 首先证明一个辅助问题的 NP 完全性, 再利用该问题证明另一个问题的 NP 完全性。最后, 以后一问题为基础, 进行多项式时间 Turing 归约, 得到上述受限 ROPAS 问题的 NP 难性。

首先说明如何修改 Johnson [18] 的一个归约, 使其符合我们的需要。该归约从团问题的一个变体出发, 归约到平衡完全二部子图问题 (参见 [31])。它接收一个具有 $2g$ 个节点和 e 条边的图, 生成一个左侧有 $\binom{g}{2} + g$ 个节点、右侧有 $e \leq \binom{2g}{2}$ 个节点的二部图。已经证明, 在原图中寻找 g 团, 等价于寻找一个左侧具有 $\binom{g}{2}$ 个节点、右侧具有同样数量节点的平衡完全二部子图。因为 $\binom{g}{2} + g \leq \binom{2g}{2}$, 可以给二部图填充孤立节点, 以统一两侧规模; 由此得到的问题是 NP 完全的。

固定比例平衡完全二部子图 (Fixed Ratio Balanced Complete Bipartite Subgraph, FIRBCBS)

实例: 数字 g , 二部图 $\mathcal{G}$, 其左侧有 $\binom{2g}{2}$ 个节点, 右侧有 $\binom{2g}{2}$ 个节点。

问题: $\mathcal{G}$ 是否包含左侧有 $\binom{g}{2}$ 个节点、右侧有 $\binom{g}{2}$ 个节点的平衡完全二部子图?

引理 4.4 ([18]). *FIRBCBS 问题是 NP 完全的。*

FIRBCBS 问题可以归约到一个辅助问题: 寻找许多个并集很小的子集。

固定比例的小并集多子集问题 (Fixed Ratio Many Subsets with Small Union, FIRMSSU)

实例: 数字 k , 集合 $\{1, \dots, 3k\}$ 的非空子集 $S_1, \dots, S_{3k}$, 其并集为 $\{1, \dots, 3k\}$ 。

问题: 这些子集中, 是否有 $2k$ 个子集的并集基数至多为 $2k$?

下面的引理推广了 [11] 中的一个早期结果。可以利用 FIRBCBS 和填充方法证明这个引理。

引理 4.5. *FIRMSSU* 问题是 *NP* 完全的。

证明. 取 FIRBCBS 的任意实例。考虑一个 $\binom{2g}{2}$ 行、 $\binom{2g}{2}$ 列的矩阵 $A = (a_{i,j})$ ，它是 $\mathcal{G}$ 的邻接矩阵的“补”：左节点 i 与右节点 j 相连时， $a_{i,j} = 0$ ；不相连时，值为 1。当且仅当该实例是肯定实例时，可以分别重排它的行和列，使左上角的 $\binom{g}{2}$ 行、 $\binom{g}{2}$ 列子矩阵只包含零。接下来，通过增加若干具有特定宽度的“条带”来填充 A （示意图 3）。取满足 $\binom{2g}{2} < k$ 的最小 k 。注意，对这样的 k ，有 $\binom{g}{2} < k$ 。令 $a = k - \binom{g}{2}$ ， $b = 3k - \binom{2g}{2} - a - k$ 。这意味着 $b > \binom{g}{2} \geq 1$ 。先增加全部由零填充的 a 行和 $a + k$ 列，再增加全部由一填充的 $k + b$ 行和 b 列。所得矩阵 A' 具有如下性质：当且仅当该实例是肯定实例时，可以分别重排行、列，使左上角的 k 行、 $2k$ 列矩形区域只包含零。因为 $b \geq 1$ ，该矩阵还至少有一行和一列只包含一。

定义一个 FIRMSSU 实例，使 A' 的每一列都成为某个子集的特征向量。这显然得到所需的实例，从而完成归约。 $\square$

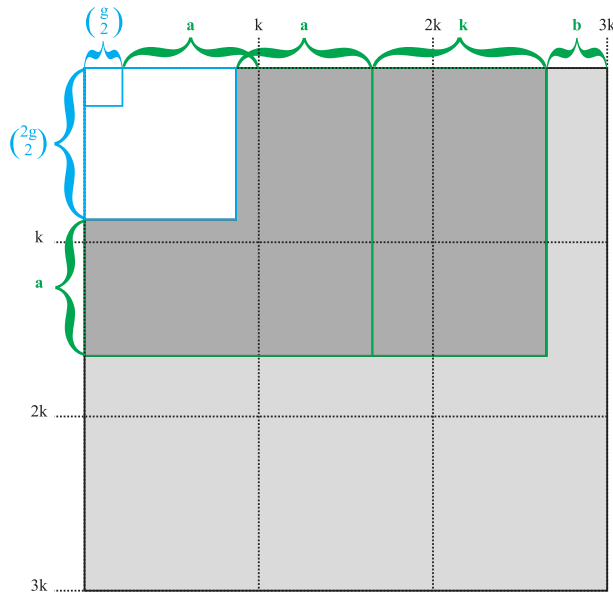


图 3: 从 FIRBCBS 到 FIRMSSU 的归约所用的填充模式；浅灰区域对应一，深灰区域对应零。

现在可以证明本小节的主要困难性结论。

定理 4.6. 当工作者被限制为两名（而 DAG 宽度可以增长）时，ROPAS 问题是 *NP* 难的。

证明. 我们给出从 FIRMSSU 到 ROPAS 的多项式时间 Turing 归约：通过检查最小期望是否足够小，可以准确判断给定的 FIRMSSU 实例是否为肯定实例。

取 FIRMSSU 的任意实例，其中有集合 $\{1, \dots, 3k\}$ 的 $n = 3k$ 个非空子集 $S_1, \dots, S_{3k}$ 。构造一个具有两名工作者的 ROPAS 实例。DAG 有三层（示例见图 4）。首先构造一个二部 DAG，其中有 n^2 个源节点 A_1 和 n 个汇节点 B_2 。源节点被划分为 n 组，每组基数为 n 。当且仅当 i 属于集合 S_j 时，从第 i 组的源节点向汇节点 j 连出一条有向边。另有 $2/3n^2$ 个额外源节点 A_2 。每个额外源节点都向每个汇节点 B_2 连出有向边。有 $2/3n$ 个内部任务 B_1 。 A_2 的每个源节点都向 B_1 的每个内部任务连出有向边。有 $1/3n^2$ 个额外汇节点 C_2 。 B_1 的每个内部任务都向 C_2 的每个额外汇节点连出有向边。第一名工作者必定能够执行 A_1 和 B_1 中的每个任务，而第二名工作者执行这些

FIRMSSU

$k=2$ $n=3k$ $S_1=\{4\}$ $S_2=\{1,2\}$ $S_3=\{3\}$ $S_4=\{2\}$ $S_5=\{5,6\}$ $S_6=\{5,6\}$

ROPAS

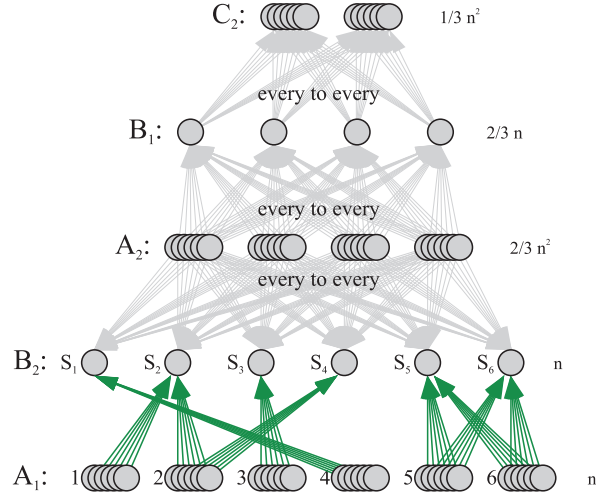


图 4: 从 FIRMSSU 到具有两名工作者、DAG 宽度不受限制的 ROPAS 的归约示例。DAG 的三层分别记为 A、B 和 C。集合 A_1 和 B_1 中的任务只能由第一名工作者执行；集合 A_2 、 B_2 和 C_2 中的任务只能由第二名工作者执行。

任务时必定失败。第二名工作者必定能够执行 A_2 、 B_2 和 C_2 中的每个任务，而第一名工作者执行这些任务时必定失败。

假设 FIRMSSU 实例是肯定实例，即存在 $2k$ 个子集，其并集大小至多为 $2k$ 。令 U 为这 $2k$ 个子集的并集；若 U 的基数小于 $2k$ ，就向 U 中加入 $\{1, \dots, 3k\}$ 的任意元素，使 U 的基数增至 $2k$ 。构造如下调度方案。第二名工作者在前 $2/3n^2$ 轮执行源节点 A_2 ，使 B_1 的 $2/3n$ 个内部任务变为可执行。与此同时，第一名工作者执行 A_1 中对应于 U 的各组里的 $2/3n^2$ 个源节点。由此， B_2 中至少 $2/3n$ 个汇节点变为可执行。在第 $2/3n^2$ 轮之后，第二名工作者开始执行 B_2 中可执行的汇节点，并在第 $2/3n^2 + 2/3n$ 轮结束前完成其中的 $2/3n$ 个。同时，第一名工作者执行 B_1 的 $2/3n$ 个内部任务，使 C_2 的全部 $1/3n^2$ 个汇节点变为可执行。在第 $2/3n^2 + 2/3n$ 轮之后，第二名工作者执行 C_2 的 $1/3n^2$ 个汇节点，而第一名工作者继续执行 A_1 中剩余的 $1/3n^2$ 个源节点。因此，在第 $n^2 + 2/3n$ 轮结束时， B_2 中尚未执行的 $1/3n$ 个汇节点已经可执行。最后，第二名工作者执行这些汇节点。计算因而在第 $n^2 + n$ 轮结束时完成。

现在假设 FIRMSSU 实例是否定实例。取任意调度方案。由于该实例是否定的，任意 $2k$ 个子集的并集基数至少为 $2k + 1$ 。每组的基数为 n ，所以要使 B_2 的 $2/3n$ 个汇节点变为可执行，必须先执行 A_1 的至少 $2/3n^2 + n$ 个源节点。因此，到第 $2/3n^2 + 2/3n$ 轮结束时，至多能执行 B_2 的 $2/3n - 1$ 个汇节点。在该轮结束前， C_2 的任何汇节点都不可能被执行，因为每个汇节点都依赖于另外 $2/3n^2 + 2/3n$ 个任务的执行。因此，到该轮结束时，至少还有 $1/3n + 1 + 1/3n^2$ 个尚未执行的任务，并且它们只能由第二名工作者执行。于是，该调度方案至少需要 $n^2 + n + 1$ 轮才能完成计算。□

4.3 使用大量工作者调度宽 DAG 的不可近似性

我们已经看到，当 DAG 可以具有很大宽度，或工作者数量可以很大时，ROPAS 都是 NP 难的。因此，有必要研究当 DAG 宽度和工作者数量都可以很大时，问题究竟有多难。这里证明，除非 $P=NP$ ，否则此时的 ROPAS 不能在小于 $5/4$ 的因子内得到近似解。

定理 4.7. 除非 $P=NP$ ，否则 ROPAS 问题无法在小于 $5/4$ 的近似因子内得到近似解。

证明. 证明构造的 DAG 与定理 4.6 证明中的 DAG 相似。由于可以使用大量工作者，在有利条件下，DAG 的执行只需四轮即可完成。条件不利时，由于部分工作者必然空闲，任何调度方案都需要五轮或更多。这就产生了所需的间隙。

具体而言，我们给出一个从 FIRMSSU 到 ROPAS 的产生间隙的归约。取 FIRMSSU 的任意实例，其中有集合 $\{1, \dots, 3k\}$ 的 $n = 3k$ 个非空子集 $S_1, \dots, S_{3k}$ 。按如下方式构造 ROPAS 实例。共有 $4k$ 名工作者。对于任意任务和任意工作者，该工作者成功执行该任务的概率都为一。DAG 有三层（示例见图 5）。共有 $6k$ 个源节点，记为 A ；它们划分为对应于数字 1 至 $3k$ 的 $3k$ 组，每组两个任务。有 $3k$ 个内部任务，记为 B_a ，同样对应于数字 1 至 $3k$ 。A 的第 i 组中的任意源节点都连接到 B_a 的内部任务 i 。此外，还有 k 个额外内部任务，记为 B_b ；A 的每个源节点都连接到 B_b 的每个内部任务。共有 $3k$ 个汇节点，记为 C_a ，分别对应于子集 S_1 至 S_{3k} 。当且仅当 i 属于子集 S_j 时， B_a 的内部任务 i 才连接到汇节点 S_j 。此外，还有 $3k$ 个额外汇节点，记为 C_b ； B_b 的每个内部任务都连接到 C_b 的每个汇节点。下面将说明，所构造 ROPAS 实例的最小期望完成时间存在一个间隙，能够区分 FIRMSSU 的肯定实例和否定实例。

假设 FIRMSSU 实例是肯定实例，即有 $2k$ 个子集的并集基数至多为 $2k$ 。我们将说明，最小期望完成时间至多为 4。令 U 为这 $2k$ 个子集的并集；若 U 的基数小于 $2k$ ，就向 U 中加入 $\{1, \dots, 3k\}$ 的任意元素，使 U 的基数增至 $2k$ 。集合 U 代表 B_a 中的 $2k$ 个内部任务。描述如下调度方案。第一轮，执行 A 中的 $4k$ 个源节点，它们的子节点是 B_a 中与 U 的任务对应的 $2k$ 个内部任务。这使与 U 对应的任务变为可执行。第二轮，执行 B_a 中这 $2k$ 个可执行任务。因为该实例是肯定实例， C_a 中的 $2k$ 个汇节点变为可执行。此外，还执行 A 中剩余的 $2k$ 个源节点。此时每个源节点都已执行，因此第二轮结束时，剩余 $2k$ 个内部任务变为可执行。第三轮，执行 C_a 中的 $2k$ 个可执行汇节点，以及 B_a 和 B_b 中的 $2k$ 个可执行内部任务。此时每个内部任务都已执行，因此所有尚未执行的汇节点都可执行。第四轮，执行剩余的 $4k$ 个汇节点。

假设 FIRMSSU 实例是否定实例，即任意 $2k$ 个子集的并集基数都是 $2k + 1$ 或更大。取任意调度方案。我们将说明，每一次执行都至少需要五轮。第一轮结束时，至多有 $2k$ 个内部任务可执行，而且它们必须属于 B_a 。这是因为 B_b 的每个任务都连接到 A 的全部 $6k$ 个源节点，而当时至多执行了其中的 $4k$ 个；同时， B_a 的每个内部任务都连接到两个不同的源节点。因此，第二轮至多能执行 B_a 中的 $2k$ 个内部任务。因为该实例是否定实例，任意 $2k$ 个子集的并集基数都是 $2k + 1$ 或更大，所以必须执行 B_a 中至少 $2k + 1$ 个任务，才能使 C_a 的 $2k$ 个汇节点变为可执行。因而，第二轮结束时， C_a 中至多有 $2k - 1$ 个汇节点变为可执行。但汇节点总数为 $6k$ ，所以当时至少有 $4k + 1$ 个汇节点尚不可执行。第三轮可以执行一些节点，但不能执行这 $4k + 1$ 个汇节点中的任何一个。即使第三轮结束时这些汇节点全部变为可执行，第四轮也至多能执行其中的 $4k$ 个，使第四轮结束时至少有一个尚未执行。因此，该调度方案至少需要五轮才能完成计算。□

致谢。 作者感谢 NASA Langley Research Center 的 Bruce R. Barkstrom 和 Argonne National Laboratory 的 Jennifer M. Schopf 对本研究的启发，并感谢 University of Massachusetts Amherst 的 Arnold L. Rosenberg 就 DAG 调度进行的讨论。“方形”中一个缺失部分的证明受到了 Xiuning Le 的启发。

FIRMSSU

$k=2$ $n=3k$ $S_1=\{4\}$ $S_2=\{1,2\}$ $S_3=\{3\}$ $S_4=\{2\}$ $S_5=\{5,6\}$ $S_6=\{5,6\}$

ROPAS

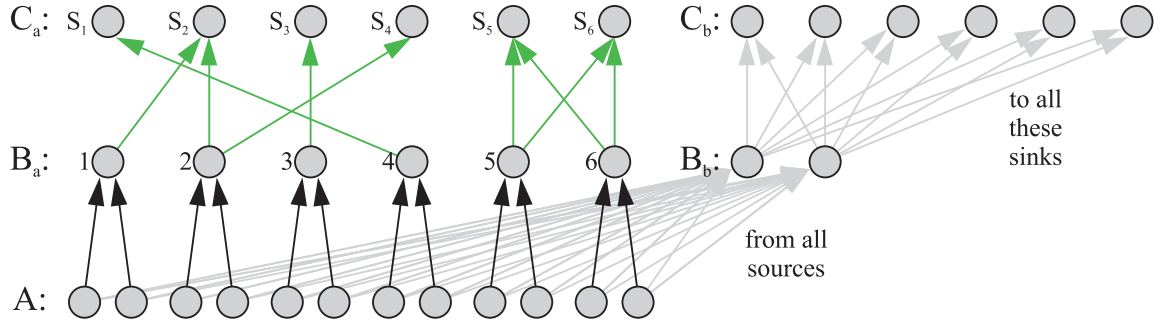


图 5: 从 FIRMSSU 到工作者数量和 DAG 宽度均不受限制的 ROPAS 的归约示例。DAG 的三层分别记为 A、B 和 C。每名工作者都可以执行任意任务。

参考文献

- [1] Annis, J., Zhao, Y., Voeckler, J., Wilde, M., Kent, S., Foster, I.: Applying Chimera Virtual Data Concepts to Cluster Finding in the Sloan Sky Survey. ACM/IEEE Conference on Supercomputing (2002) 1–14
- [2] Berriman, B., Bergou, A., Deelman, E., Good, J., Jacob, J., Katz, D., Kesselman, C., Laity, A., Singh, G., Su, M.-H., Williams, R.: Montage: A Grid-Enabled Image Mosaic Service for the NVO. Astronomical Data Analysis Software & Systems (2003)
- [3] Cordasco, G., Malewicz, G., Rosenberg, A.: Advances in IC-Scheduling Theory: Scheduling Expansive and Reductive Dags and Scheduling Dags via Duality. IEEE Transactions on Parallel and Distributed Systems (2007) to appear
- [4] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms (2nd Edition). MIT Press (2001)
- [5] Crescenzi, P., Kann, V. (eds.): A compendium of NP optimization problems. <http://www.nada.kth.se/~viggo/wwwcompendium/node173.html>
- [6] Deelman, E., Kesselman, C., Mehta, G., Meshkat, L., Pearlman, L., Blackburn, K., Ehrens, P., Lazzarini, A., Williams, R., Koranda, S.: GriPhyN and LIGO, building a virtual data grid for gravitational wave scientists. High Performance Distributed Computing (HPDC'02) (2002) 225–234.
- [7] Dilworth, R.P.: A decomposition theorem for partially ordered sets. Annals of Mathematics, Vol. 51 (1950) 161–166
- [8] Fernandez, A., Armacost, R., Pet-Edwards, J.: Understanding Simulation Solutions to Resource constrained Project Scheduling Problems with Stochastic task Durations. Engineering Management Journal, Vol. 10(4) (1998) 5–13

- [9] Fernandez, A., Armacost, R.L., Pet-Edwards, J.: A Model for the Resource Constrained Project Scheduling Problem with Stochastic Task Durations. 7th Industrial Engineering Research Conference Proceedings (1998)
- [10] Foster, I., Kesselman, C. [eds.]: The Grid: Blueprint for a New Computing Infrastructure, 2nd ed. Morgan-Kaufmann, San Francisco, CA (2004)
- [11] Gao, L., Malewicz, G.: Toward Maximizing the Quality of Results of Dependent Tasks Computed Unreliably. *Theory of Computing Systems* (2007) available online
- [12] Garey, M.R., Johnson, D.S.: *Computers and Intractability*. Freeman, New York (1979)
- [13] Goel, A., Indyk, P.: Stochastic load balancing and related problems. 40th Annual Symposium on Foundations of Computer Science (FOCS) (1999) 579–586
- [14] Hillier, F.S., Lieberman, G.J.: *Introduction to Operations Research*, 8th ed. McGraw-Hill (2004)
- [15] Herroelen, W., Leus, R.: Project scheduling under uncertainty: Survey and research potentials. *European Journal of Operational Research*, Vol. 165(2) (2005) 289–306
- [16] Horn, J.V., Dobson, J., Woodward, J., Wilde, M., Zhao, Y., Voeckler, J., Foster, I.: *Grid-Based Computing and the Future of Neuroscience Computation*. *Methods in Mind*, MIT Press, (2005).
- [17] Ireland, K., Rosen, M.: *A Classical Introduction to Modern Number Theory*, 2nd ed. Springer-Verlag (1990)
- [18] Johnson, D.S.: The NP-completeness column: An ongoing guide. *Journal of Algorithms*, Vol. 8(3) (1987) 438–448
- [19] Kleinberg, J., Rabani, Y., Tardos, E.: Allocating Bandwidth for Bursty Connections. *SIAM Journal on Computing*, Vol. 30(1) (2000) 191–217
- [20] Lin, G., Rajaraman, R.: Approximation Algorithms for Multiprocessor Scheduling under Uncertainty. 19th ACM Symposium on Parallelism in Algorithms and Architectures (2007) to appear
- [21] Lin, G.: New paradigms and approximation algorithms for optimization under uncertainty. Ph.D. Dissertation, College of Computer and Information Science, Northeastern University (2006)
- [22] Malewicz, G.: Implementation and Experiments with an Algorithm for Parallel Scheduling of Complex Dags under Uncertainty. Submitted for publication. Preliminary version 8th Workshop on Algorithm Engineering and Experiments (ALENEX'06) (2006) 66–74
- [23] Malewicz, G., Foster, I., Rosenberg, A., Wilde, M.: A Tool for Prioritizing DAGMan Jobs and Its Evaluation. *Journal of Grid Computing* Vol. 5(2) (2007) 197–212
- [24] Malewicz, G., Rosenberg, A.L.: On batch-scheduling dags for Internet-based computing. 11th European Conference on Parallel Processing (Euro-Par) (2005) 262–271

- [25] Malewicz, G., Rosenberg, A., Yurkewych, M.: Toward a Theory for Scheduling Dags in Internet-Based Computing. *IEEE Transactions on Computers*, Vol. 55(6) (2006) 757–768
- [26] Microsoft Project 2003 <http://www.microsoft.com/office/project/default.asp>
- [27] Mori, M., Tseng, C.: A Resource Constrained Project Scheduling Problem with Reattempt at Failure: A Heuristic Approach. *Journal of the Operations Research Society of Japan*, Vol. 40(1) (1997) 33–44
- [28] Narasimhan, M., Ramanujam, J.: A fast approach to computing exact solutions to the resource-constrained scheduling problem. *ACM Transactions on Design Automation of Electronic Systems*, Vol. 6(4) (2001) 490–500
- [29] Norris, J.R.: *Markov Chain*. Cambridge University Press (1997)
- [30] Özdamar, L., Ulusoy, G.: A survey on the resource-constrained project scheduling problem. *IIE Transactions*, Vol. 27 (1995) 574–586
- [31] Peeters, R.: The maximum edge biclique problem is NP-complete. *Discrete Applied Mathematics*, Vol. 131(3) (2003) 651–654
- [32] Rosenberg, A.L.: On scheduling mesh-structured computations for Internet-based computing. *IEEE Transactions on Computers*, Vol. 53 (2004) 1176–1186
- [33] Rosenberg, A.L., Yurkewych, M.: Guidelines for scheduling some common computation-dags for Internet-based computing. *IEEE Transactions on Computers*, Vol. 54(4) (2005) 428–438
- [34] Sarmenta, L.F.G.: Sabotage-tolerance mechanisms for volunteer computing systems. *Future Generation Computer Systems*, Vol. 18(4) (2002) 561–572
- [35] Skutella, M., Uetz, M.: Scheduling precedence-constrained jobs with stochastic processing times on parallel machines. *12th ACM-SIAM Symposium on Discrete Algorithms (SODA)* (2001) 589–590
- [36] Thain D., Tannenbaum, T., Livny, M.: Distributed computing in practice: the Condor experience. *Concurrency and Computation: Practice and Experience*, Vol. 17 (2005) 323–356
- [37] Tseng, C.C., Mori, M., Yajima, Y.: A project scheduling model considering the success probability. *Proc. of the Association of Asian Pacific Operational Research Societies (APROS)* (1994) 399–406
- [38] Turnquist, M.A., Nozick, L.K.: Allocating Time and Resources in Project Management Under Uncertainty. *36th Annual Hawaii International Conference on System Sciences (HICSS)* (2003) 250c